

Image Filtering using Fourier Transform

Fourier Transform

1) DFT
DFT stands for Discrete Fourier Transform. It is a mathematical tool that transforms a finite-length discrete-time sequence into a discrete-frequency sequence of the same length.

For a complex sequence (x[0], x[1], ..., x[N-1]) of length N, the DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N - 1$$

Where:
- X[k] is the k-th frequency component (frequency domain representation),
- j is the imaginary unit,
- N is the sequence length.

2) FFT
The FFT exploits the symmetry and periodicity of the exponential factors $e^{-j\frac{2\pi}{N}kn}$ in the DFT formula. By recursively breaking down a DFT into smaller DFTs, the number of computations is drastically reduced.

Apply 2D Fourier Filtering to a 2D Grayscale Image

Image → Frequency Domain → Multiply by Filter → Back to Image

1) Grayscale Image as a 2D Signal
A grayscale image is simply a matrix: f(x,y) with size N×M.
Each pixel represents an intensity value (typically 0–255 or floating-point). Mathematically, it is just a 2D discrete signal.

2) Compute the 2D DFT to Obtain the Frequency Spectrum F(u,v)

$$F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y) \cdot e^{-j2\pi(\frac{xu}{N} + \frac{yv}{M})}$$

- F(u,v) is complex-valued (has magnitude and phase).
- The magnitude tells how strong a frequency component is.
- The phase encodes structural and spatial information.

3) Frequency-domain shift
By default, the DC component (low frequency) appears in the corner of the spectrum. For filter design, it is much easier if the low frequency is at the center.
Before computing the DFT, multiply the image by: $(-1)^{x+y}$. This is mathematically equivalent to shifting the spectrum to the center.

4) Use Frequency-Domain Filter (H(u,v))
A frequency filter is simply a mask with the same size as the spectrum.
Filtering is done by pointwise multiplication: $G(u,v) = F(u,v) \cdot H(u,v)$

- Common filter types:
- Low-pass Filter (Blur / Denoise): Keeps low frequencies (center region), Suppresses high frequencies
 - High-pass Filter (Edge Enhancement): Suppresses low frequencies, Keeps high frequencies.
 - Band-pass: Keeps a specific frequency range.Enhances textures.
 - Band-stop: Removes a specific frequency band. Useful for removing structured noise patterns.
 - Notch Filter: Removing stripe noise and periodic interference
 - Periodic noise in images appears as pairs of bright symmetric points in the spectrum. removes those specific frequency locations.

5) Output filtered image using Inverse DFT (IDFT)
• The imaginary part after IDFT should be nearly zero, Use the real part of the result.

$$g(x,y) = \frac{1}{NM} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} G(u,v) \cdot e^{j2\pi(\frac{xu}{N} + \frac{yv}{M})}$$

The GPU Workflow

- img_to_complex_shift
 - Convert image to complex and apply centering shift.
- Row DFT pass #1
 - Perform 1D DFT on each row (naturally contiguous → coalesced).
- Transpose
- Row DFT pass #2
 - Perform 1D DFT on each row of the transposed matrix (which corresponds to column DFT of original).
- Transpose back
- apply_mask_kernel
- Inverse process: transpose → row IDFT → transpose → row IDFT
- complex_to_img_scale_invshift
 - Fused final kernel: real extraction + scaling + inverse shift

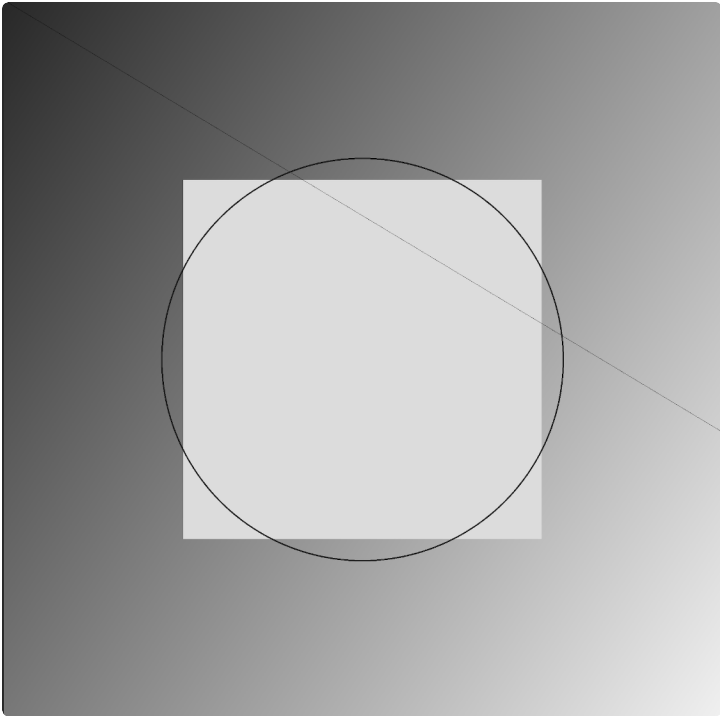
Algorithm execution time (ms)	lowpass	highpass	bandpass	auto-notch
DFT in cpu	108014.7776	114964.3412	115416.3875	113494.5547
FFT in cpu	684.5656	661.4348	690.1388	868.5792
DFT in gpu	1824.0382	1827.4294	1827.7997	2025.8535
FFT in gpu	40.0649	40.1767	40.1161	235.4811

Optimization Approach

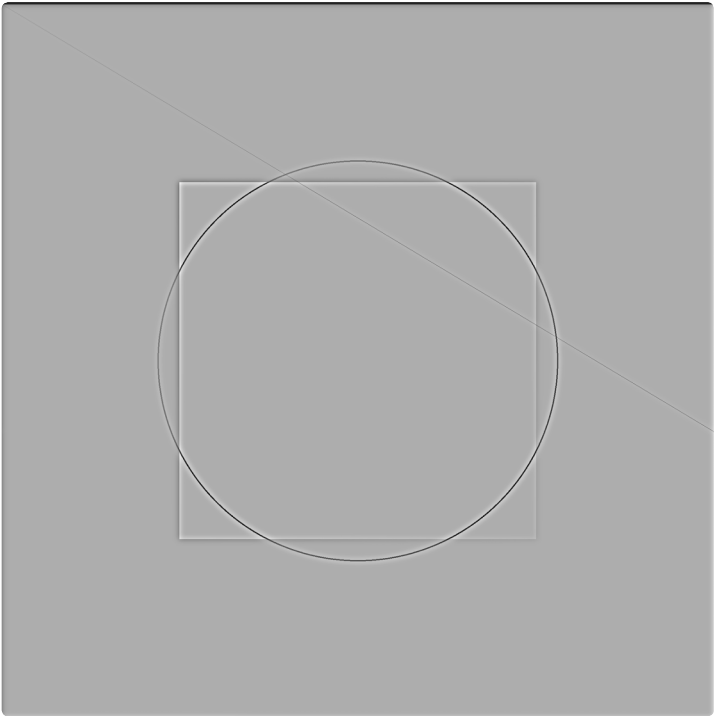
- Converting column DFT into row DFT via transpose to ensure coalesced memory access
 - Instead of performing column DFT directly (which would cause strided memory access), the GPU Converts column DFT into row DFT on a transposed matrix. This ensures both 1D DFT passes use contiguous memory access.
- Using shared-memory tiled transpose with padding to avoid bank conflicts
 - Each block processes a 32×32 tile
 - Read tile from global memory into shared memory (coalesced), then Synchronize
 - Write transposed tile back to global memory (coalesced)
 - Why Padding (TILE_DIM + 1) : Shared memory is divided into memory banks. If multiple threads access the same bank simultaneously → bank conflict → serialization. Transposed access patterns often cause bank conflicts.
- Fusing scaling and inverse shift into the final kernel to reduce global memory traffic and overhead kernel launches
 - After IDFT, CPU runs another loop: 1 Extract real part, 2 Multiply by 1/(N*N), 3 Apply inverse shift, 4 Store result
 - Instead GPU splits these into multiple kernels, it used a single fused kernel

Algorithm Comparison	lowpass	highpass	bandpass	auto-notch
abs(DFTcpu-FFTcpu)/FFTcpu	156.7859	172.8105	166.2365	129.6669
abs(DFTcpu-DFTgpu)/DFTgpu	58.2174	61.9104	62.1450	55.0231
abs(FFTcpu-FFTgpu)/FFTgpu	16.0864	15.4631	16.2035	2.6885
abs(DFTcpu-FFTgpu)/FFTgpu	2694.9952	2860.4680	2876.0590	480.9688

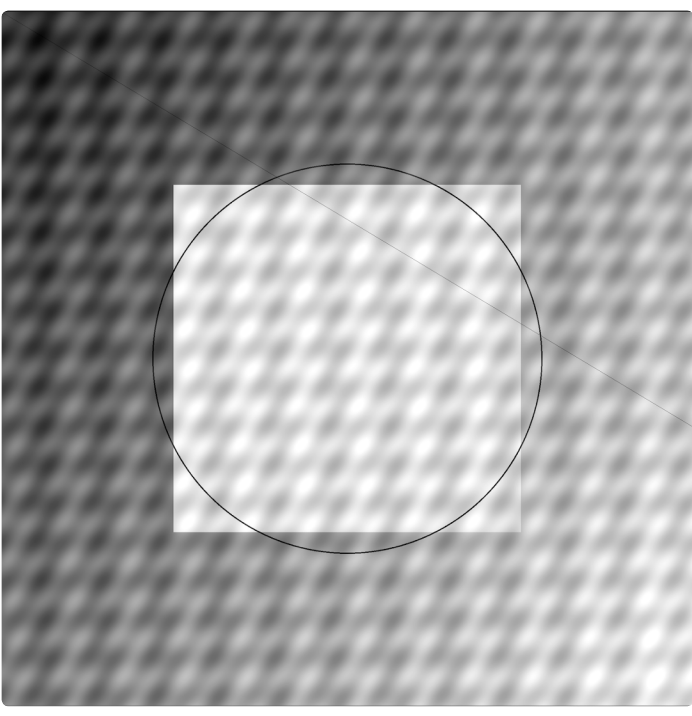
Image Result



Original



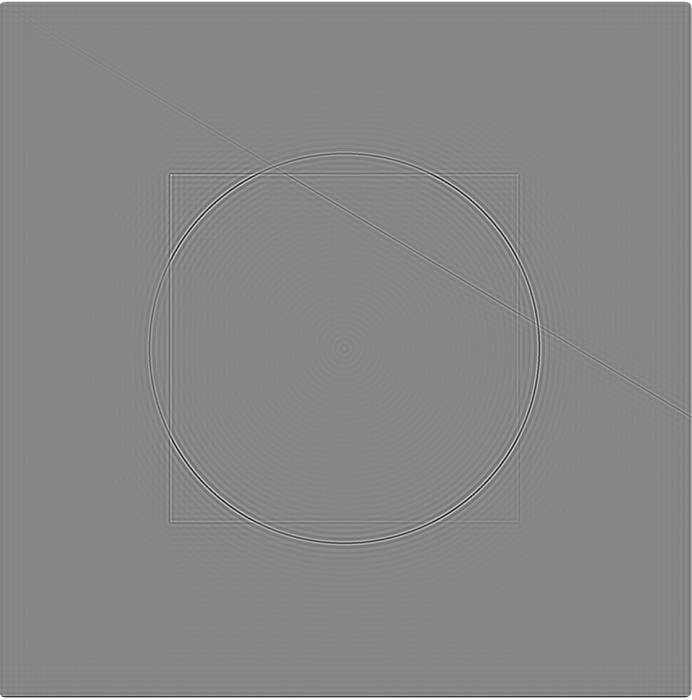
Highpass



Original + Periodic Noise



Lowpass



Bandpass



Auto-Notch Denoise